Architectural Clarification and Blockchain Interaction Model

Phase-1 Blockchain E-Voting System

The Phase-1 specification describes a blockchain-based voting architecture where administrative configuration, voter management, eligibility verification, and election lifecycle management are handled by a centralized backend, while the final vote record is stored on a blockchain ledger. The document correctly establishes the overall structure of the voting system, including voter identity anchoring, eligibility verification through Merkle trees, election deployment through smart contracts, and blockchain-based vote recording. In the original design, however, the sequence of actions surrounding blockchain transactions suggests that the backend system triggers and submits vote transactions to the blockchain network. This interpretation conflicts with the requirement that votes must be cryptographically signed by the voter themselves through a wallet interface. Because blockchain voting systems rely heavily on cryptographic ownership and non-repudiation, the responsibility for signing the vote transaction must reside with the voter's wallet rather than with the backend infrastructure.

The system therefore requires a modification in the interaction layer between the application frontend, backend services, and the blockchain network. The remainder of the system — including voter registration, administrative workflows, eligibility engineering, Merkle root construction, candidate management, election configuration, and database storage — can remain exactly as defined in the original specification. The primary adjustment concerns how vote transactions are created, signed, transmitted to the blockchain, and recorded within the system.

Change 1: Transaction Origination and Signing Responsibility

In the original architecture the system flow indicates that the platform "triggers the blockchain transaction" once the voter confirms their candidate selection. This phrasing implies that the backend service acts as the entity that sends the transaction to the blockchain network. In a blockchain-based voting system, however, such a design would undermine one of the fundamental advantages of decentralized ledgers: the guarantee that each action is cryptographically authorized by the entity that owns the blockchain account performing the action.

To preserve this property, the vote transaction must originate from the frontend application and be signed by the voter's wallet. The frontend acts as the interface between the user and the blockchain, and the wallet (such as MetaMask or WalletConnect) performs the cryptographic signing process. When the voter selects a candidate and confirms their choice, the frontend invokes the smart contract's voting function through a Web3 provider. The wallet then prompts the voter to approve the transaction and sign it using their private key. Once the voter confirms the signature, the transaction is broadcast directly to the blockchain network and recorded on the ledger.

Under this corrected architecture the backend no longer signs or submits voting transactions. Instead, it remains responsible for authentication, voter management, election configuration, and data storage, while the blockchain interaction for voting occurs directly between the frontend wallet and the smart contract. This adjustment ensures that every vote is cryptographically tied to the voter's blockchain address and cannot be forged or submitted by an intermediary system.

Change 2: Responsibility for Double-Voting Prevention

The original document includes a "pre-ballot check" in which the backend verifies that the voter has not already cast a vote before allowing the ballot screen to appear. While this check is useful for user-interface feedback, it cannot serve as the authoritative mechanism for preventing duplicate voting. In a decentralized voting system the ultimate source of truth must always be the smart contract running on the blockchain.

The responsibility for enforcing the "one voter, one vote" rule must therefore reside inside the smart contract itself. Each voting contract maintains a mapping between blockchain addresses and their voting status. When a transaction is submitted, the contract checks whether the address that initiated the transaction has already voted. If the address has previously participated, the transaction fails and the vote is rejected automatically by the contract logic. Because this verification occurs on-chain, it cannot be bypassed by any off-chain system or backend service.

The backend's role in this process becomes supportive rather than authoritative. The backend may check whether a voter appears to have voted according to its database records, and it may display warnings or update the user interface accordingly, but the definitive enforcement of voting limits occurs within the smart contract. This ensures that even if the backend is compromised or malfunctioning, the blockchain itself prevents duplicate voting.

Change 3: Transaction Recording and Backend Synchronization

In the original flow the backend appears to initiate the blockchain transaction and therefore receives the transaction hash directly from the blockchain interaction service. When vote transactions are signed and submitted by the frontend wallet instead, the mechanism for recording transaction data must change slightly.

After the voter signs and submits the transaction, the blockchain network generates a transaction hash that uniquely identifies the vote record. The frontend application receives this hash immediately after the transaction is broadcast. The frontend must then send the transaction hash to the backend through an API endpoint. The backend stores this information along with the voter ID, election ID, wallet address, and timestamp. This stored data allows the platform to provide a user interface showing confirmation of voting, maintain an auditable record linking system users to blockchain transactions, and display transaction links for independent verification on a block explorer.

In this architecture the blockchain remains the immutable source of the vote record, while the backend maintains a synchronized off-chain record for application functionality and administrative auditing. The backend database does not determine the vote outcome but simply tracks metadata associated with blockchain transactions.

Final System Architecture

With the above changes implemented, the final architecture of the Phase-1 system becomes a hybrid model combining centralized application management with decentralized vote recording. The backend continues to serve as the operational core of the platform, handling voter registration, Aadhaar-based identity verification, OTP authentication, election creation, candidate management, eligibility configuration, and Merkle tree generation. The backend also deploys the election smart contract once the administrative configuration is finalized, since contract deployment is an administrative action rather than a voter action. When deployment occurs, the backend records the contract address and associates it with the election record in the database.

The frontend functions as the interactive interface through which voters participate in elections. It retrieves election data from the backend, displays the ballot interface, and manages the connection to the user’s blockchain wallet. When a voter decides to cast a vote, the frontend retrieves the relevant election parameters, including the smart contract address and the Merkle proof required for eligibility verification. Using a Web3 provider, the frontend calls the voting

function of the deployed smart contract. The wallet then prompts the voter to sign the transaction. After the signature is approved, the transaction is transmitted directly to the blockchain network where the smart contract verifies eligibility, checks whether the voter has already participated, and increments the vote count for the selected candidate.

Once the transaction has been submitted, the frontend receives the resulting transaction hash and sends it to the backend for recordkeeping. The backend stores this information and uses it to update the voter dashboard, confirm successful participation, and provide transparency tools such as links to blockchain explorers where the transaction can be independently verified. The final vote tally remains entirely determined by the blockchain ledger, ensuring immutability and transparency while the backend supports usability, administrative control, and system coordination.

Through this architecture the system achieves a balanced design: administrative processes and identity verification remain manageable within a traditional backend infrastructure, while the actual act of voting becomes a cryptographically signed action performed directly by the voter on the blockchain. This separation of responsibilities preserves decentralization where it matters most — the recording of votes — while maintaining the usability and organizational control necessary for a practical election management platform.

Clarification of Smart Contract Interaction in the Phase-1 Blockchain E-Voting Architecture

The Phase-1 specification of the blockchain-based e-voting system establishes a hybrid architecture in which a traditional backend infrastructure manages voter identity, election configuration, and administrative workflows, while the blockchain ledger serves as the immutable layer that records votes and guarantees transparency. In the original design document, the administrative flows, voter lifecycle, eligibility verification mechanisms, and smart contract deployment procedures are correctly described. However, the interaction between the system components and the smart contract requires a clearer definition in order to preserve the fundamental principles of blockchain security, particularly the requirement that vote transactions must be signed by the voter.

A blockchain system derives its trust guarantees from cryptographic ownership of accounts. Every transaction written to the blockchain must be signed using the private key of the account that initiates the action. In a voting system this requirement is critical, because it ensures that a vote cannot be created or altered by any intermediary such as the server infrastructure or system administrators. Therefore, although the backend continues to manage most aspects of the election platform, the act of casting a vote must originate from the voter's wallet through the frontend application. This section clarifies how smart contracts are called within the system and explains the division of responsibilities between the frontend, backend, and blockchain layers.

Smart Contract Invocation During Vote Casting

The act of voting is the only operation in the system that must be cryptographically authorized by the voter themselves. For this reason, the voting transaction is initiated directly by the frontend application rather than by the backend server. When the voter selects a candidate and confirms their ballot, the frontend application communicates with the blockchain using a Web3 provider connected to the voter's wallet, such as MetaMask or another compatible wallet interface.

At this point the frontend loads the deployed election contract using the contract's address and ABI, both of which were previously stored in the backend database when the election was deployed. The frontend then calls the voting function defined in the smart contract. Because this call represents a blockchain transaction, the wallet prompts the voter to approve and sign the operation using their private key. Only after the voter confirms this signature does the transaction get broadcast to the blockchain network.

Once the transaction reaches the blockchain, the smart contract executes the voting logic. The contract verifies that the election is currently active according to the defined start and end

timestamps. It checks whether the voter has already participated by consulting the on-chain mapping that records which addresses have voted. It also verifies the voter's eligibility using the Merkle proof provided by the frontend. If all verification conditions are satisfied, the contract increments the vote count for the selected candidate and marks the voter's address as having voted. The vote is then permanently recorded on the blockchain ledger.

This architecture guarantees that every vote is tied to the wallet address that signed the transaction. Because the backend never possesses the voter's private key, it cannot generate or modify votes on behalf of users. This design preserves the non-repudiation property of blockchain systems and ensures that the integrity of the election is enforced by the smart contract itself.

Smart Contract Interaction During Election Deployment

While voting must originate from the voter's wallet, not all interactions with the smart contract are performed by the frontend. Administrative actions such as deploying a new election contract are performed by the backend system. In the Phase-1 architecture, election creation is managed by an administrator through the backend dashboard. After the administrator finalizes the election configuration — including candidate details, election timing, and eligibility parameters such as the Merkle root — the backend initiates the deployment of the smart contract.

The backend calls a blockchain interaction service that compiles and deploys the contract to the chosen blockchain network. The contract constructor receives the required initialization parameters, such as the list of candidate identifiers, the election start and end times, and the eligibility proof representing the authorized voter set. Once the blockchain confirms successful deployment, it returns a unique contract address that identifies the newly created election contract.

The backend records this contract address in the elections table of the system database. From that moment onward, the contract address serves as the reference point through which both the frontend and backend interact with the election logic. The administrator does not need to manage a personal wallet or manually deploy contracts, because the backend infrastructure handles this process automatically. This approach keeps administrative operations manageable while preserving the decentralized nature of vote recording.

Smart Contract Interaction for Reading Election Results

In addition to voting and contract deployment, the system also interacts with the smart contract to retrieve election data such as vote counts and election status. These operations are read-only queries that do not modify the blockchain state. Because read operations do not require cryptographic signatures or gas fees, they can be performed directly by the frontend without requiring wallet approval.

When users view the results of an election, the frontend queries the smart contract using functions such as a results retrieval method defined within the contract. The contract returns the current vote counts for each candidate, allowing the frontend to display the tally in the user interface. Because these values are retrieved directly from the blockchain, they reflect the immutable record maintained by the distributed ledger rather than relying on backend calculations.

This read-only interaction allows the system to present transparent and verifiable election outcomes while maintaining efficiency. The backend may also retrieve this data for administrative reporting, but the blockchain remains the definitive source of truth for all vote counts.

Final Interaction Architecture of the System

With these clarifications in place, the final architecture of the Phase-1 e-voting system becomes a hybrid structure that divides responsibilities among three layers. The backend remains responsible for managing application logic, identity verification, voter registration, election configuration, candidate management, and smart contract deployment. The frontend acts as the interactive layer through which users view elections, connect their wallets, and sign transactions. The blockchain network hosts the election smart contracts and serves as the immutable ledger that records votes and enforces voting rules.

Administrative actions flow from the administrator through the backend to the blockchain, allowing the backend to deploy election contracts and manage election configuration. Voting actions originate from the voter through the frontend wallet interface and proceed directly to the blockchain, ensuring that each vote is signed by the voter’s own cryptographic key. Read-only queries, such as retrieving results or election status, can be performed directly by the frontend through blockchain calls that do not require wallet signatures.

Through this division of responsibilities, the system maintains both usability and security. The backend provides the centralized coordination necessary for managing elections and verifying voter identities, while the blockchain ensures that vote recording remains decentralized, transparent, and tamper-proof. This architecture preserves the integrity of the voting process while allowing the application to remain practical and manageable within a real-world deployment environment.

Gas Fees and Transaction Costs in the Blockchain E-Voting System

The blockchain e-voting system relies on a smart contract deployed on a blockchain network to record votes and enforce election rules. Whenever a blockchain network processes a transaction that modifies its state, computational resources are consumed by the nodes that maintain the network. To compensate these nodes and prevent malicious actors from flooding the network with meaningless transactions, blockchain systems require users to pay a fee known as **gas**. Gas represents the computational effort required to execute operations on the blockchain, and it is paid using the native cryptocurrency of the network.

Understanding gas fees is important for designing the architecture of the voting system because every interaction with the smart contract does not incur the same cost. Some actions require gas because they change blockchain data, while others are read-only operations that do not modify the ledger and therefore do not require payment. The design of the voting system must clearly separate these two categories of operations.

Why Gas Fees Exist in Blockchain Systems

Blockchain networks operate without a central authority and rely on independent nodes to validate and record transactions. These nodes perform several tasks whenever a transaction occurs. They verify cryptographic signatures, execute smart contract logic, update the distributed ledger, and store the resulting state changes permanently. Because these tasks require computation, storage, and network resources, the blockchain protocol introduces gas fees to compensate validators for processing transactions.

Gas fees also serve as a protection mechanism against network abuse. If transactions were completely free, malicious actors could generate enormous numbers of meaningless transactions that would overload the network and disrupt legitimate activity. By attaching a cost to each transaction, the blockchain discourages spam and ensures that only transactions with genuine purpose are submitted.

For the e-voting system, gas fees therefore represent the cost of executing voting logic on the blockchain and permanently recording election outcomes.

Operations That Require Gas

Gas fees are required whenever a transaction modifies the state of the blockchain. In the Phase-1 voting system, several operations fall into this category because they change data stored inside the smart contract.

One of the most significant operations requiring gas is the **deployment of the election smart contract**. When an administrator finalizes the election configuration, the backend deploys a new smart contract to the blockchain. This deployment writes the compiled contract code and its initial parameters—such as the election start time, end time, candidate identifiers, and eligibility proof—into the blockchain's storage. Because storing contract code consumes significant computational resources, contract deployment is usually the most expensive transaction in the entire system.

Another operation that requires gas is the **casting of a vote**. When a voter signs a transaction through their wallet and calls the contract's voting function, the smart contract performs several tasks. It verifies that the election is currently active according to the stored timestamps. It checks whether the voter's blockchain address has already participated by consulting the participation mapping stored in the contract. It verifies the voter's eligibility using the Merkle proof supplied with the transaction. Finally, it increments the vote counter associated with the chosen candidate and marks the voter's address as having voted. Each of these operations modifies contract storage, and therefore the transaction consumes gas.

In some implementations, the smart contract may also contain an administrative function that explicitly **ends the election** once the voting period is complete. If such a function updates the election state or locks the contract to prevent further votes, calling it also requires gas because it changes the blockchain's stored data.

All of these actions share one characteristic: they write new information to the blockchain ledger. Whenever this occurs, a transaction must be executed and a gas fee must be paid.

Operations That Do Not Require Gas

Not every interaction with the blockchain involves writing data. Many operations simply read information from the smart contract without modifying the blockchain state. These operations are known as **read-only calls** and do not require gas fees.

In the voting system, read-only calls are used to retrieve information such as the current vote counts for each candidate, the election's start and end times, or whether a particular wallet address has already voted. When the frontend application displays election results or shows the status of an election, it typically performs these types of read operations.

Because read-only calls do not create transactions, they do not need to be validated by blockchain nodes in the same way as state-changing operations. Instead, the data is simply retrieved from the existing blockchain state and returned to the requesting application. As a result, users can view election results or verify transaction information without paying any gas fees.

This distinction between read operations and write operations is important because it ensures that only essential actions—such as casting votes or deploying contracts—incur costs on the network.

Managing Gas Costs During Development

In a real blockchain environment, gas fees are paid using the network's native cryptocurrency. For example, transactions on the Ethereum network require payment in Ether. If the voting system were deployed directly on a public blockchain, each vote transaction would require the voter to pay a small amount of cryptocurrency to cover the computational cost of recording the vote.

For a development or academic project, however, using real cryptocurrency is unnecessary and impractical. Instead, developers typically build and test their applications using simulated blockchain environments or public testing networks.

One common approach is to use a **local blockchain network** provided by development tools such as Hardhat or Ganache. A local blockchain runs entirely on the developer's machine and simulates the behavior of a real blockchain network. These tools automatically create test accounts funded with large amounts of simulated cryptocurrency, allowing developers to deploy contracts and submit transactions without spending real money. Although gas is still calculated internally by the system, it is paid using fake tokens that exist only within the local environment.

Another approach is to deploy the system on a **public blockchain test network**. Test networks behave like real blockchain networks but use tokens that have no financial value. Developers can obtain these tokens from online faucet services and use them to pay gas for testing transactions. Deploying the voting contract on a test network allows the system to operate in a realistic environment where transactions are validated by distributed nodes and recorded on a publicly accessible ledger.

Both approaches allow the e-voting system to demonstrate full blockchain functionality without incurring real financial costs.